

# Express + Mongoose — REST API Lab

MERN Stack Bootcamp · Week 5 · Day 23 · Companion to Lecture Day 23

**Duration:** ~45 minutes | **Prerequisite:** Day 22 lab (shop\_db with products) | **Tools:** Node.js, Thunder Client (or Postman)

---

## Lab overview

Build **shop-api**, an Express server that uses **Mongoose** to read and write the products collection you created on Day 22. You will implement a full REST API and test every endpoint before optionally connecting your React app.

| Phase | Task                                | Time    |
|-------|-------------------------------------|---------|
| 0     | Scaffold project & install packages | 5 min   |
| 1     | Connect Mongoose + Product model    | 8 min   |
| 2     | GET routes (list + by id)           | 10 min  |
| 3     | POST + PUT routes                   | 12 min  |
| 4     | DELETE + CORS + test all endpoints  | 10 min  |
|       | Total                               | ~45 min |

**Phase 0** Scaffold project

5 min

### Step 0.1 — Create folder and package.json

```
mkdir shop-api && cd shop-api
npm init -y
npm pkg set type=module
```

### Step 0.2 — Install dependencies

```
npm install express mongoose dotenv cors
```

### Step 0.3 — Dev dependency (auto-restart)

```
npm install -D nodemon
```

### Step 0.4 — Create .env and .gitignore

```
# .env
PORT=5000
MONGO_URI=mongodb+srv://USER:PASS@cluster.mongodb.net/shop_db

# .gitignore
node_modules/
.env
```

### Step 0.5 — package.json scripts

```
"scripts": {
  "start": "node server.js",
  "dev": "nodemon server.js"
}
```

**Checkpoint:** `npm run dev` will start after `server.js` exists.

## Phase 1 Mongoose connection & Product model

8 min

### Step 1.1 — *config/db.js*

```
import mongoose from "mongoose";

export async function connectDB() {
  await mongoose.connect(process.env.MONGO_URI);
  console.log("MongoDB connected");
}
```

### Step 1.2 — *models/Product.js*

```
import mongoose from "mongoose";

const productSchema = new mongoose.Schema(
  {
    title: { type: String, required: true, trim: true },
    price: { type: Number, required: true, min: 0 },
    category: { type: String, default: "general" },
    inStock: { type: Boolean, default: true },
    tags: [String],
  },
  { timestamps: true }
);

export const Product = mongoose.model("Product", productSchema);
```

### Step 1.3 — *server.js (minimal)*

```
import "dotenv/config";
import express from "express";
import cors from "cors";
import { connectDB } from "../config/db.js";

const app = express();
app.use(cors());
app.use(express.json());

await connectDB();

app.get("/", (req, res) => res.json({ message: "shop-api running" }));

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server on port ${PORT}`));
```

**Checkpoint:** Browser `http://localhost:5000` shows `{"message": "shop-api running"}`.  
Terminal shows MongoDB connected.

## Phase 2 GET routes

10 min

### Step 2.1 — *controllers/productController.js*

```
import { Product } from "../models/Product.js";

export const getProducts = async (req, res) => {
  try {
```

```

    const filter = req.query.category ? { category: req.query.category } : {};
    const products = await Product.find(filter);
    res.json(products);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

export const getProductById = async (req, res) => {
  try {
    const product = await Product.findById(req.params.id);
    if (!product) return res.status(404).json({ message: "Product not found" });
    res.json(product);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

```

### Step 2.2 — routes/productRoutes.js

```

import { Router } from "express";
import { getProducts, getProductById } from "../controllers/productController.js";

const router = Router();
router.get("/", getProducts);
router.get("/:id", getProductById);
export default router;

```

### Step 2.3 — Mount routes in server.js

```

import productRoutes from "./routes/productRoutes.js";
app.use("/api/products", productRoutes);

```

### Step 2.4 — Test in Thunder Client

- GET http://localhost:5000/api/products → array of products from Day 22
- Copy one `_id` value
- GET http://localhost:5000/api/products/PASTE\_ID\_HERE → single object
- GET http://localhost:5000/api/products?category=accessories → filtered list

**Checkpoint:** List returns your seeded products. Single GET returns 404 for fake IDs.

**Phase 3**      **POST & PUT routes**

12 min

### Step 3.1 — Add createProduct and updateProduct to controller

```

export const createProduct = async (req, res) => {
  try {
    const product = await Product.create(req.body);
    res.status(201).json(product);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
};

export const updateProduct = async (req, res) => {
  try {
    const product = await Product.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );
  }
};

```

```
    if (!product) return res.status(404).json({ message: "Product not found" });  
    res.json(product);  
  } catch (err) {  
    res.status(400).json({ message: err.message });  
  }  
};
```

### **Step 3.2 — Register routes**

```
router.post('/', createProduct);  
router.put('/:id', updateProduct);
```

### **Step 3.3 — Test POST (JSON body)**

POST http://localhost:5000/api/products  
Content-Type: application/json

```
{  
  "title": "Webcam HD",  
  "price": 59.99,  
  "category": "accessories",  
  "inStock": true,  
  "tags": ["video"]  
}
```

### **Step 3.4 — Test PUT**

PUT http://localhost:5000/api/products/PASTE\_ID  
{ "price": 49.99, "onSale": true }

**Checkpoint:** POST returns 201 with new `_id`. Verify in Compass. PUT returns updated document.

**Step 4.1 — deleteProduct in controller**

```
export const deleteProduct = async (req, res) => {
  try {
    const product = await Product.findByIdAndDelete(req.params.id);
    if (!product) return res.status(404).json({ message: "Product not found" });
    res.json({ message: "Deleted", id: product._id });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};
router.delete('/:id', deleteProduct);
```

**Step 4.2 — Optional 404 catch-all (server.js, after routes)**

```
app.use((req, res) => {
  res.status(404).json({ message: "Route not found" });
});
```

**Step 4.3 — Full endpoint checklist**

- GET /api/products — 200, array
- GET /api/products/:id — 200 or 404
- POST /api/products — 201 with valid body; 400 if title missing
- PUT /api/products/:id — 200 or 404
- DELETE /api/products/:id — 200 or 404

**Checkpoint:** All five methods work. DELETE removes doc from Compass.

**Bonus — Connect React (if time permits)**

```
// In Products.jsx – replace DummyJSON URL
const res = await fetch("http://localhost:5000/api/products");
const products = await res.json();
// Note: your API returns an array directly, not { products: [...] }
```

Ensure cors() is enabled and both servers run: npm run dev in shop-api and React app on port 5173.

**Submission checklist**

- Screenshot of GET /api/products showing JSON array in Thunder Client or browser
- Screenshot of successful POST (201) or PUT (200)
- GitHub repo link (private) with shop-api — no .env committed
- One sentence: What does runValidators: true do on update?

**Troubleshooting**

| Problem                  | Fix                                                                |
|--------------------------|--------------------------------------------------------------------|
| MongoDB connection error | Check MONGO_URI, password URL-encoding, Atlas IP whitelist.        |
| Empty array on GET       | Wrong database in URI; confirm shop_db and products in Compass.    |
| Cannot POST / 404        | Mount <code>app.use('/api/products', routes)</code> before listen. |

|                         |                                                               |
|-------------------------|---------------------------------------------------------------|
| CORS error in React     | Add <code>app.use(cors());</code> ; restart API server.       |
| Cast to ObjectId failed | Invalid :id in URL; copy full <code>_id</code> from GET list. |